Project Part 2

AI Personal Finance Assistant

System Design

Group G

Sofia Ursachi

Aidan Gillooley

Adam Hayden

Ulysse Couchoud

Jamie Ocallaghan

0. Group Members:

Student name	Student ID	Main responsibility worked on	Approximate contribution %
Aidan Gillooley aidan.gillooley2@mail.dcu.ie	A00030921	Infrastructure & Software Requirements	20%
Adam Hayden adam.hayden22@mail.dcu.ie	A23448102	Migration & Submission	20%
Ulysse Couchoud ulysse.couchoud2@mail.dcu.ie	A00133493	Web Mock-up and System Architecture Diagram	20%
Jamie Ocallaghan jamie.ocallaghan28@mail.dcu.ie	A23365913	Data Security	20%
Sofia Ursachi sofia.ursachi2@mail.dcu.ie	A00035832	Platform & Architecture Description and Personal Data Handling	20%

1. Web Mock-up

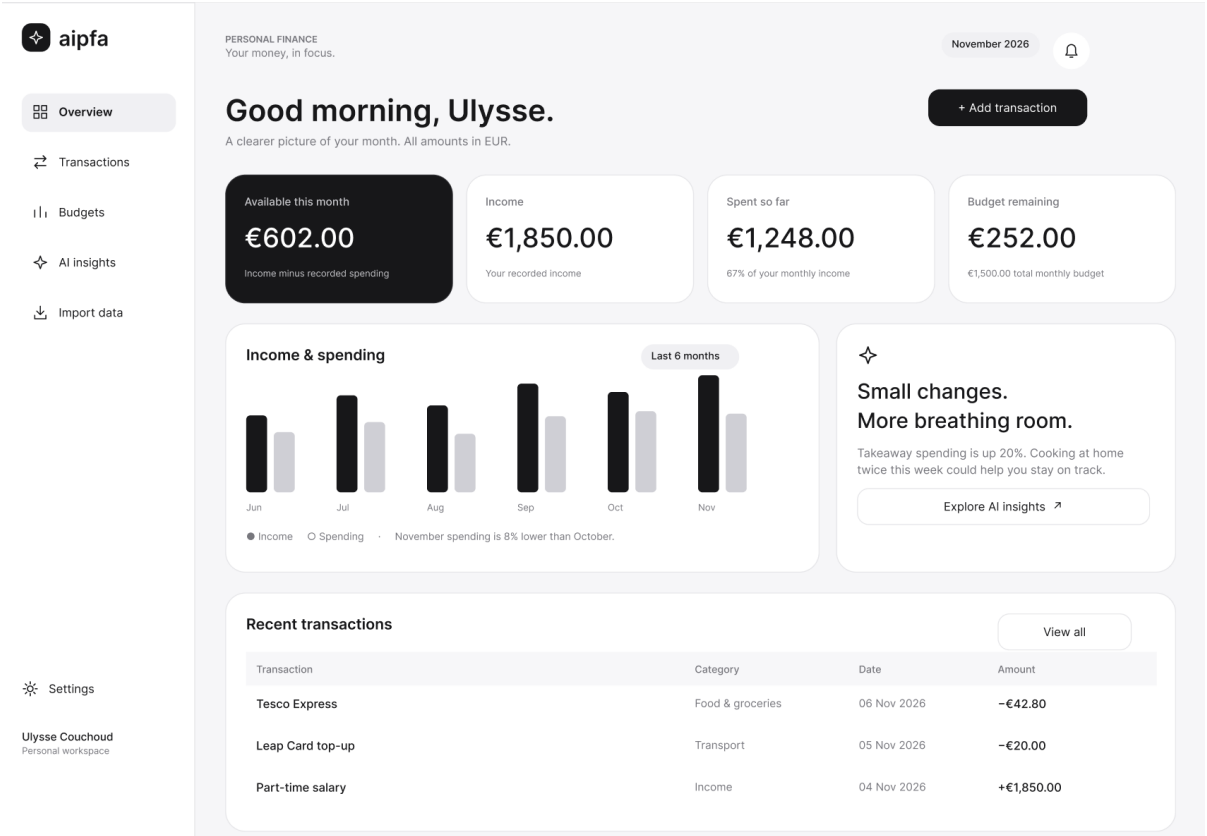
The mock-up below shows the main screens of our AI Personal Finance Assistant. We designed it in Figma to plan the layout, navigation and user flow before development. It focuses on a clean and simple interface, so users can quickly see their financial situation and talk to the AI assistant without feeling overwhelmed.

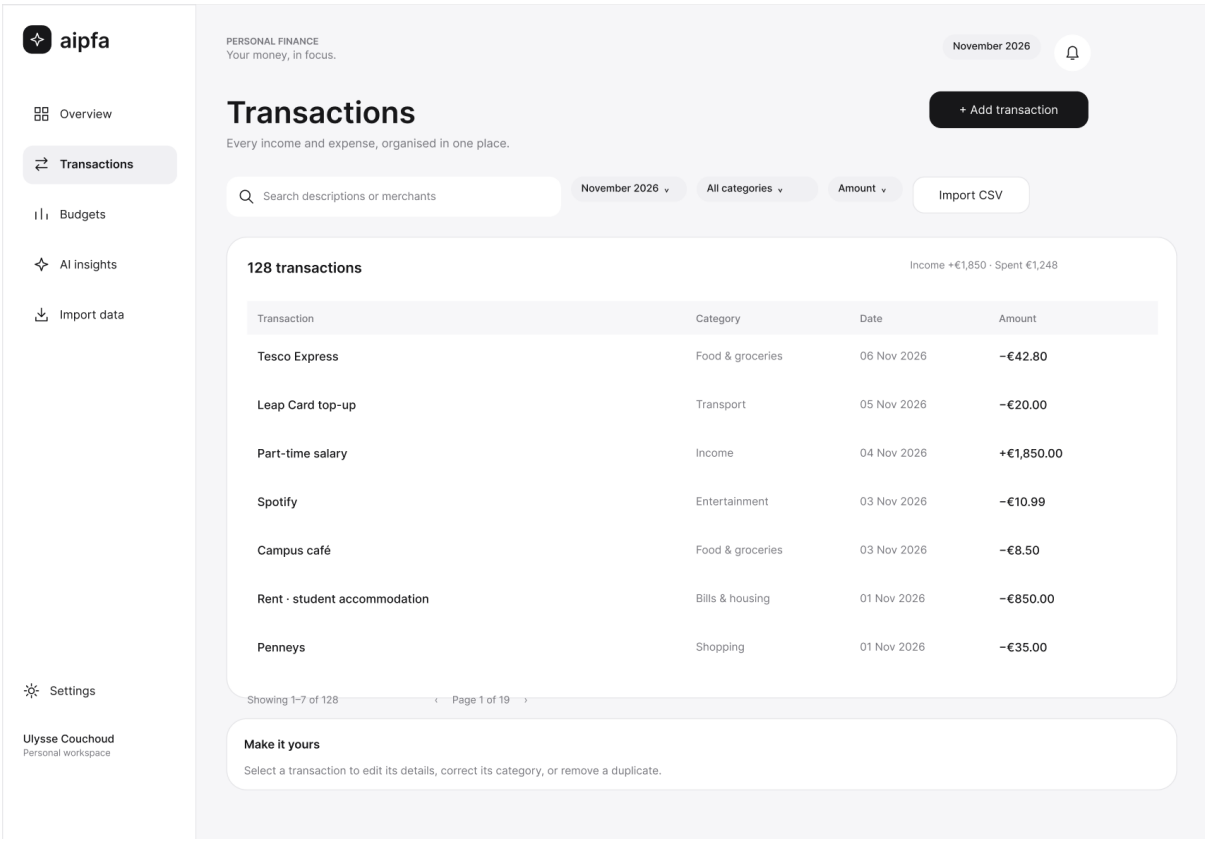
The screenshots show the key parts of the application: the dashboard with an overview of balances and spending, the transactions and budget pages where users can track and organise their expenses, and the AI assistant, which gives personalised advice and answers questions about the user's finances.

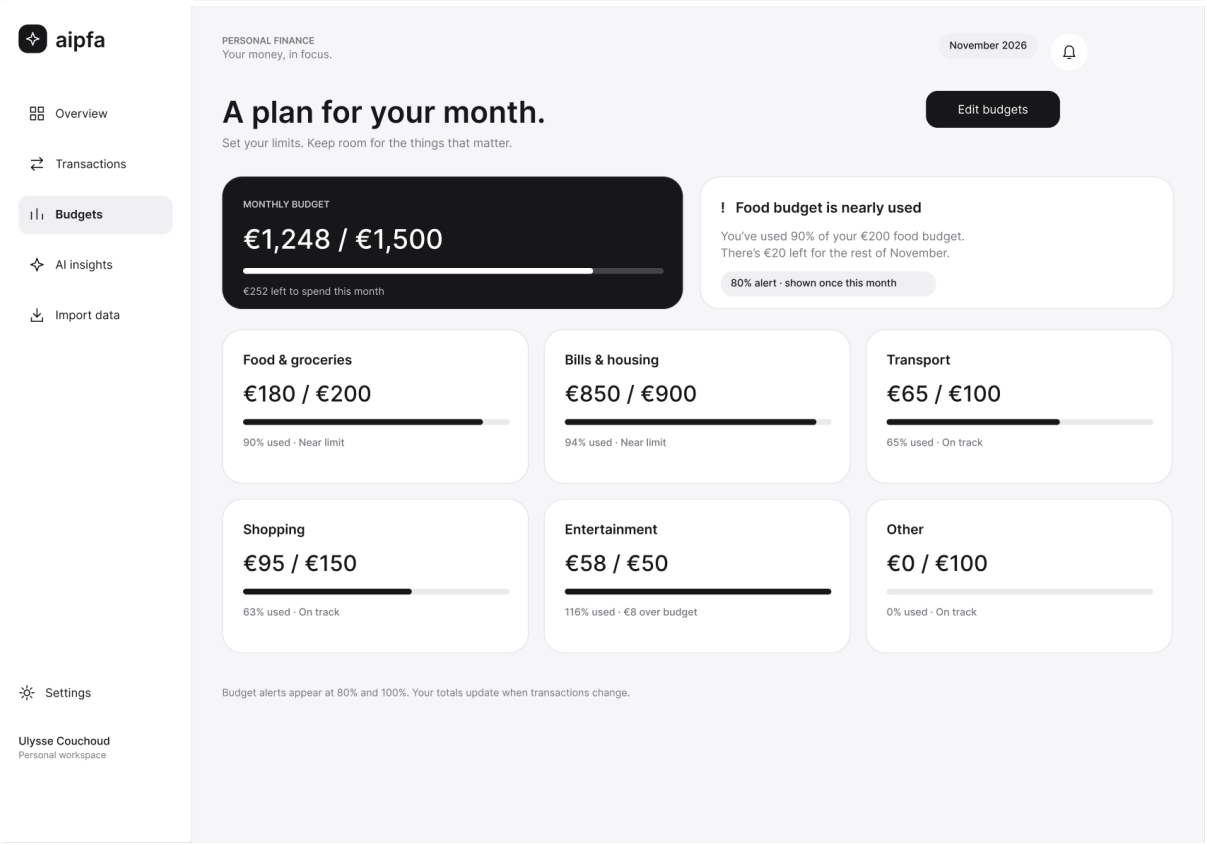
The full version, including every screen, is available through the Figma link above.

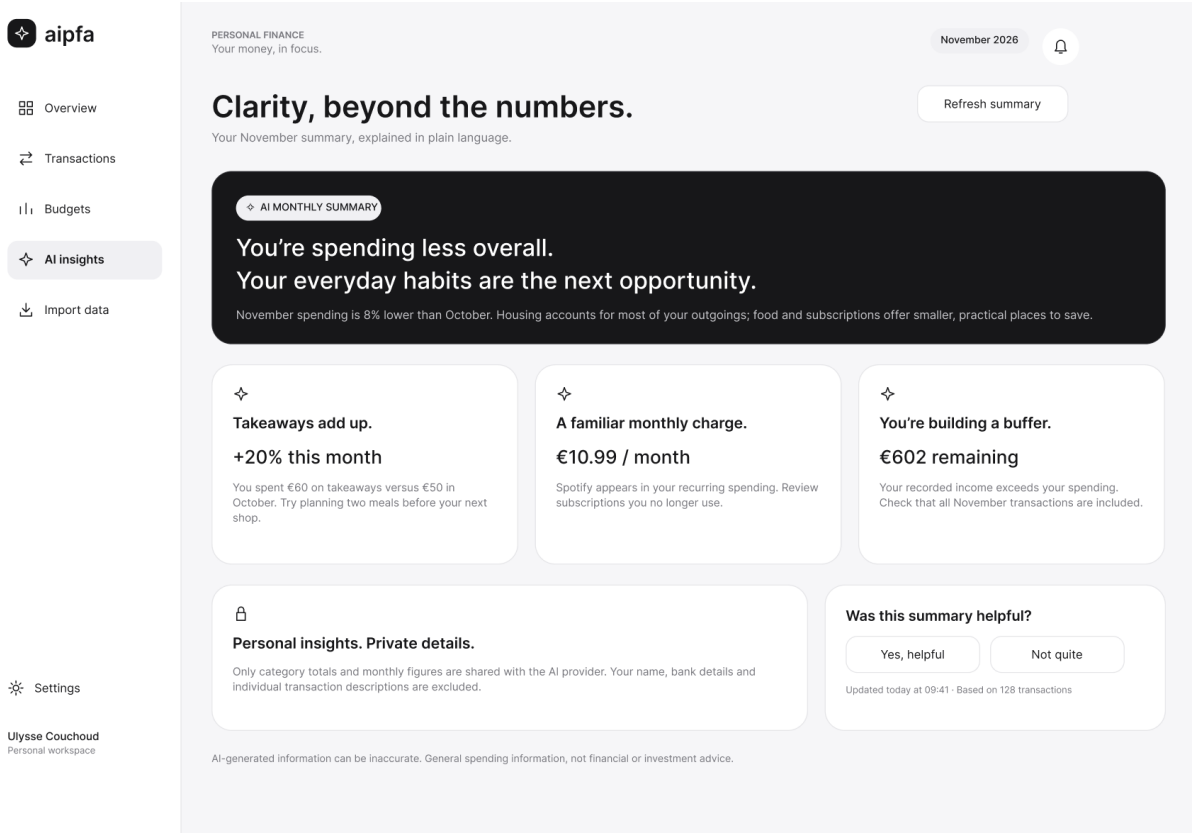
Figma mock-up

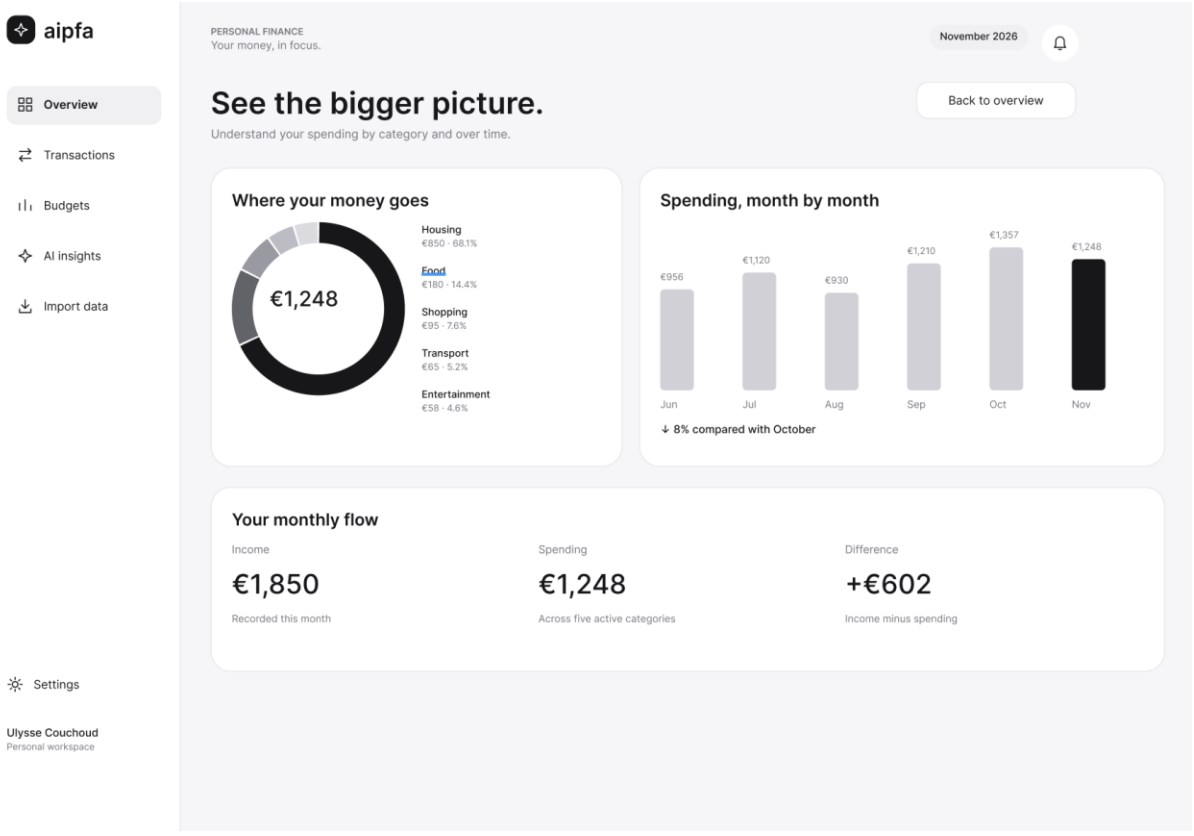
Relevant screenshots:




aipfa
Overview
Transactions
Budgets
AI insights
Import data
Settings
Ulysse Couchoud
Personal workspace
PERSONAL FINANCE
Your money, in focus.
November 2026
Transactions
Every income and expense, organised in one place.
+ Add transaction
Search descriptions or merchants
November 2026
All categories
Amount
Import CSV
128 transactions
Income +€1,850 · Spent €1,248
Transaction
Category
Date
Amount
Tesco Express
Food & groceries
06 Nov 2026
−€42.80
Leap Card top-up
Transport
05 Nov 2026
−€20.00
Part-time salary
Income
04 Nov 2026
+€1,850.00
Spotify
Entertainment
03 Nov 2026
−€10.99
Campus café
Food & groceries
03 Nov 2026
−€8.50
Rent · student accommodation
Bills & housing
01 Nov 2026
−€850.00
Penneys
Shopping
01 Nov 2026
−€35.00
Showing 1–7 of 128
Page 1 of 19
Make it yours
Select a transaction to edit its details, correct its category, or remove a duplicate.


aipfa
Overview
Transactions
Budgets
AI insights
Import data
Settings
Ulysse Couchoud
Personal workspace
PERSONAL FINANCE
Your money, in focus.
November 2026
A plan for your month.
Set your limits. Keep room for the things that matter.
Edit budgets
MONTHLY BUDGET
€1,248 / €1,500
€252 left to spend this month
! Food budget is nearly used
You've used 90% of your €200 food budget.
There's €20 left for the rest of November.
80% alert · shown once this month
Food & groceries
€180 / €200
90% used · Near limit
Bills & housing
€850 / €900
94% used · Near limit
Transport
€65 / €100
65% used · On track
Shopping
€95 / €150
63% used · On track
Entertainment
€58 / €50
116% used · €8 over budget
Other
€0 / €100
0% used · On track
Budget alerts appear at 80% and 100%. Your totals update when transactions change.

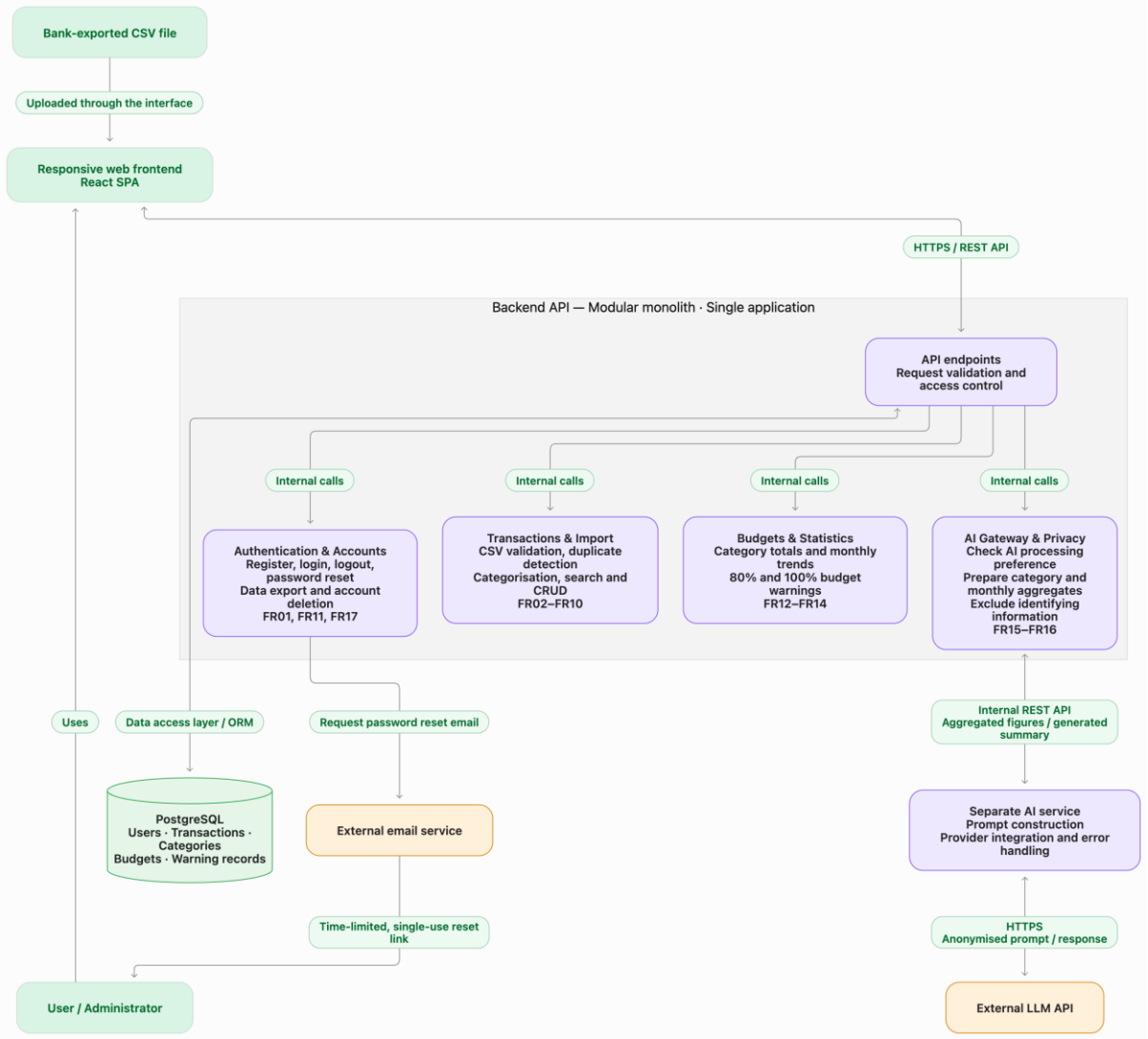

aipfa
Overview
Transactions
Budgets
AI insights
Import data
Settings
Ulysse Couchoud
Personal workspace
PERSONAL FINANCE
Your money, in focus.
November 2026
Clarity, beyond the numbers.
Your November summary, explained in plain language.
Refresh summary
AI MONTHLY SUMMARY
You're spending less overall.
Your everyday habits are the next opportunity.
November spending is 8% lower than October. Housing accounts for most of your outgoings; food and subscriptions offer smaller, practical places to save.
Takeaways add up.
+20% this month
You spent €60 on takeaways versus €50 in October. Try planning two meals before your next shop.
A familiar monthly charge.
€10.99 / month
Spotify appears in your recurring spending. Review subscriptions you no longer use.
You're building a buffer.
€602 remaining
Your recorded income exceeds your spending. Check that all November transactions are included.
Personal insights. Private details.
Only category totals and monthly figures are shared with the AI provider. Your name, bank details and individual transaction descriptions are excluded.
Was this summary helpful?
Yes, helpful
Not quite
Updated today at 09:41 · Based on 128 transactions
AI-generated information can be inaccurate. General spending information, not financial or investment advice.


aipfa
Overview
Transactions
Budgets
AI insights
Import data
Settings
Ulysse Couchoud
Personal workspace
PERSONAL FINANCE
Your money, in focus.
November 2026
See the bigger picture.
Understand your spending by category and over time.
Back to overview
Where your money goes
€1,248
Housing
€850 · 68.1%
Food
€180 · 14.4%
Shopping
€95 · 7.6%
Transport
€65 · 5.2%
Entertainment
€58 · 4.6%
Spending, month by month
€956
€1,120
€930
€1,210
€1,357
€1,248
Jun
Jul
Aug
Sep
Oct
Nov
↓ 8% compared with October
Your monthly flow
Income
€1,850
Recorded this month
Spending
€1,248
Across five active categories
Difference
+€602
Income minus spending

1. System Architecture Diagram

Architecture and Components

The system uses a modular monolith for the core backend, a PostgreSQL database and a separate AI service. The responsive React frontend communicates with the backend through an HTTPS REST API.



Purple identifies custom modules, green reused software and orange external services. Backend modules run within one application. Only aggregated financial figures are passed to the separate AI service.

Mermaid diagram

Component	Purpose
Web frontend	Provides authentication screens, transaction management, CSV upload, budgets, charts and AI insights.
Authentication and accounts	Handles registration, login, password reset, data export and account deletion. Restricts users to their own data.
Transactions and imports	Validates manual entries and CSV files, detects duplicates, assigns categories and supports searching, editing and deletion.
Budgets and statistics	Calculates category totals and monthly trends, compares spending with budgets and generates warnings at 80% and 100%.
AI Gateway and separate AI service	The gateway prepares aggregates and excludes identifying information. The AI service calls the LLM and handles provider errors.
PostgreSQL	Stores users, transactions, categories, budgets and warning records.
External services	The LLM generates spending explanations; the email service delivers password reset links.

Architecture Choices

A modular monolith reduces deployment, testing and communication complexity for a five-person team working within one semester. Microservices would allow independent scaling, but require additional deployments, network communication and distributed error handling. The trade-off is that the core modules initially deploy and scale together.

The AI service remains separate so its provider can change independently. If it becomes unavailable, transactions, budgets and dashboards remain usable. Financial calculations are performed by the backend; the LLM only explains the supplied figures.

Provider Comparison

Decision	Comparison and proposed choice
Cloud: Render vs AWS Lightsail	Render provides managed application hosting, private services and PostgreSQL. Lightsail offers greater server control but requires more administration when using a virtual server. **Render is proposed** to reduce operational work, with backend services and the database in Frankfurt. Paid services must be budgeted: free PostgreSQL expires after 30 days, and private services have no free instance option.
LLM: OpenAI vs Google Gemini	OpenAI does not use API data for training by default; Gemini Paid Services also exclude prompts and responses from product improvement. Both retain some data for operational or safety purposes. **OpenAI is the proposed initial provider**, subject to testing summary accuracy, latency and cost against Gemini. A

	provider adapter allows replacement without changing the core backend.

These comparisons use the providers' official documentation.

Reasons for Third-Party Components

Component	Reason for reuse
React and shadcn/ui	Reusable interface components reduce development time and maintain visual consistency.
Chart library	Produces spending and trend visualisations without implementing chart rendering manually.
PostgreSQL and ORM	Relational storage supports consistent financial records; the ORM simplifies queries and schema migrations.
JWT library	Provides established token signing and verification.
Password-hashing library	Stores salted password hashes instead of plain-text passwords.
Validation and CSV libraries	Check inputs and parse quoted fields, delimiters and escaped characters reliably.
LLM API	Generates summaries without hosting a language model.
Email service	Delivers reset messages without operating a mail server.
Cloud hosting	Provides deployment infrastructure while reducing infrastructure work for the team.

2. Platform & Architecture Description

Summary: We propose **Render (Frankfurt, EU)** and a **modular monolith with one separate AI service**. This keeps cost and admin work low for a small team, keeps data in the EU, and lets the AI provider change independently.

Cloud provider

	Render	**AWS/ Azure/ Google**
EU hosting	Frankfurt (only EU region)	Several EU regions
Managed PostgreSQL	Yes	Yes (RDS, Azure DB, Cloud SQL)
Effort to run	Low: deploy from Git, TLS handled	Higher: networks, IAM, load balancers
Pricing	Fixed monthly per service	Pay-per-use, harder to predict

- **Why Render:** a five-person start-up team cannot spend time on server administration, and predictable pricing suits a freemium business.
- **Trade-off:** fewer services than AWS. Standard PostgreSQL and containers mean we can move to AWS later if needed.

Architecture pattern

- **Chosen – modular monolith + AI service:** one codebase and deployment for core features; the AI service is separate so it can fail or change provider without affecting the rest.
- **SOA:** not a full SOA – a service bus shared across many systems is too heavy for one product. We borrow its idea for the AI service, which sits behind a defined interface, so our design is a **hybrid**.
- **Rejected:** microservices (many deployments and network calls for a small team); serverless (cold starts, and long CSV imports exceed function time limits).

Trade-offs considered

- **Latency:** core modules call each other in-process; AI insights load separately so dashboards never wait for the LLM.
- **Security:** only the frontend and API are public; the AI service and database are private (Section 5).
- **Cost and maintenance:** two services, one database, one pipeline; a busy module (e.g. CSV imports) can be split out later.

3. Infrastructure & Software Requirements

Summary: AIPFA is a two-service app (modular monolith + separate AI service) on Render (Frankfurt, EU) with managed PostgreSQL. No on-premises hardware is required — production is fully managed; the team uses existing workstations.

4.1 Hardware

On premises: none. Fully cloud hosted.

Developer workstations: standard laptops running Node.js, Git, editor, optional Docker for local PostgreSQL parity. No GPU (LLM is API-called, not hosted).

Client: any modern browser (desktop/mobile); lightweight React SPA, no heavy client compute.

4.2 Compute & Capacity (proposed targets — confirm current Render plan specs)

Component	Resource	Rationale
Frontend (React SPA)	Render static site, CDN-backed	No server compute; low latency
Backend API (monolith)	1 web service, ~0.5–1 vCPU / 512 MB–2 GB RAM	Small team, low launch traffic; in-process calls
PostgreSQL	Managed Starter tier, ~256 MB–1 GB storage	Small dataset: users, transactions, budgets, warnings
AI service	Separate small web service	Isolated so LLM latency/failure never blocks dashboards
Email	External API	No mail server to run

Scaling: vertical plan upgrades first; horizontal scaling of the stateless (JWT) backend only if needed. CSV imports are the heaviest workload and can later be split into a worker service. Read replicas not justified at this scale.

4.3 Networking

Public: frontend + backend API only, over HTTPS/TLS (Render-managed).

Private: AI service and PostgreSQL not publicly exposed.

Region: Frankfurt (EU) — data stays in EU; compute co-located with DB.

No VPC/load-balancer setup required (deliberate trade-off: less control, minimal admin).

4.4 Availability & Resilience

Managed TLS, auto-restart, Git-based redeploys.

AI isolation: if LLM/AI service fails, transactions, budgets and dashboards remain usable.

Backups: Render managed PostgreSQL backup/point-in-time recovery (confirm plan retention).

Proposed targets: RPO ≤ 24 h, RTO ≤ a few hours. Single-region — acceptable for MVP; multi-region deferred until cost-justified.

4.5 Development Tools

Git/GitLab · Jira (Agile) · Figma · Render CI/CD (auto-deploy on push + tests) · Docker (optional local parity) · unit/integration/API/CSV-security testing and UAT.

4.6 Software Stack & Dependencies

Frontend: React SPA, shadcn/ui, chart library; HTTPS REST to backend.

Backend: JWT + bcrypt auth; password reset via email service (time-limited single-use links); validation + CSV parsing libraries; in-process budget/statistics calculations; AI Gateway anonymises/aggregates before calling LLM.

Data: ORM over PostgreSQL (also mitigates SQL injection).

Secrets: DB credentials and LLM keys in Render environment variables (never in code/repo).

Third parties: Render (hosting/DB — standard PostgreSQL + containers, portable to AWS); LLM API (OpenAI initially, adapter for Gemini — graceful degradation); email service (retry/queue on failure).

4.7 Cost & Maintainability

Fixed monthly per-service pricing suits a freemium start-up; two services, one DB, one pipeline keeps overhead low. Budget from launch: free PostgreSQL expires after 30 days

and private services have no free tier. Standard PostgreSQL/containers allow a future move to AWS without redesign.

5. Data Security

As our AI Personal Finance Assistant will store users' personal and financial information, it is important that we have security measures in place to protect this data. We want users to feel confident that their information is safe and that nobody else can access their financial records.

Main Security Risks and Prevention

There are several security risks that we need to consider when developing our application:

- **Unauthorised access:** Someone could try to access another user's financial information. To prevent this, users must log in before accessing the system. Passwords will be securely hashed using bcrypt, and JWT authentication will be used. We will also ensure that every user can view and manage only their own transactions.
- **SQL injection and harmful files:** Users could enter harmful information into forms or upload unsafe CSV files. To reduce this risk, all information entered and files uploaded will be checked and validated before being processed. We will also use an ORM to help protect against SQL injection attacks.
- **Data being intercepted:** As users will be accessing their finances online, there is a risk that information could be intercepted while being transferred. We will use HTTPS/TLS encryption to protect information being sent between the frontend and backend.
- **Passwords and API keys being exposed:** Database passwords and LLM API keys must be kept secure. These will be stored using Render environment variables rather than being written directly into our code or GitLab repository.
- **AI data exposure:** Our application uses an external LLM to provide spending summaries and suggestions. To protect users, the AI Gateway will remove personal identifying information and only send anonymised or aggregated financial figures. Information such as names, email addresses, passwords and bank details will never be sent to the external AI service.

Cloud Security and Monitoring

As we have chosen Render to host our application and PostgreSQL database in Frankfurt, we will use the security features available through the platform. Database access will be restricted so that it is not unnecessarily exposed to the public. Access to the Render dashboard will also be protected using two-factor authentication, and only authorised team members will have access to the system.

We will follow the OWASP Top 10 guidelines, which highlight common security risks for web applications. We will also use the NIST Cybersecurity Framework as a guide when identifying risks and deciding how to prevent and respond to attacks. Regular testing will be carried out on the login system, API endpoints, database access and CSV uploads to identify possible weaknesses.

Handling a Security Breach

If a security breach occurs, we will first try to identify the problem and restrict access to any affected parts of the system. Any compromised passwords or API keys will be changed, and the issue will be investigated to understand how it happened. Database backups will be used where necessary to restore lost or damaged information. If personal data is affected, users and the relevant authorities will be informed where required by GDPR.

By putting these measures in place, we aim to reduce the chances of a security breach and protect the financial information stored within our application.

6. Personal Data Handling

Summary: AIPFA collects only the data needed to run the account and analyse spending. It is stored in the EU, only the user can see their financial records, and users can export or delete everything at any time.

Data	Why needed	Who can access	Kept for
Name, Email	Login, password reset	User, email service (resets only)	Until account deleted
Password	Login	Bcrypt hash only	Until account deleted
Transactions, budgets	Core features	User only	Until user deletes them
Bank CSV files	Import	Backend, during processing	Discarded after parsing
Spending totals for AI	AI insights	AI service, LLM – no identifiers	Not stored by us
Logs, backups	Security, recovery	Authorised admins	90/30 days

GDPR in practice

- **Lawful basis:** contract for core features; consent for AI insights, which can be turned off.
- **Purpose limitation:** data is never sold, used for ads or used to train AI models.
- **Security:** TLS, encryption at rest, per-user access checks; admins cannot view users' finances (Section 5).
- **Transfers:** only anonymised totals go to the US-based LLM, covered by Standard Contractual Clauses.
- **Accountability:** DPIA before launch; processing agreements with hosting, LLM and email providers.

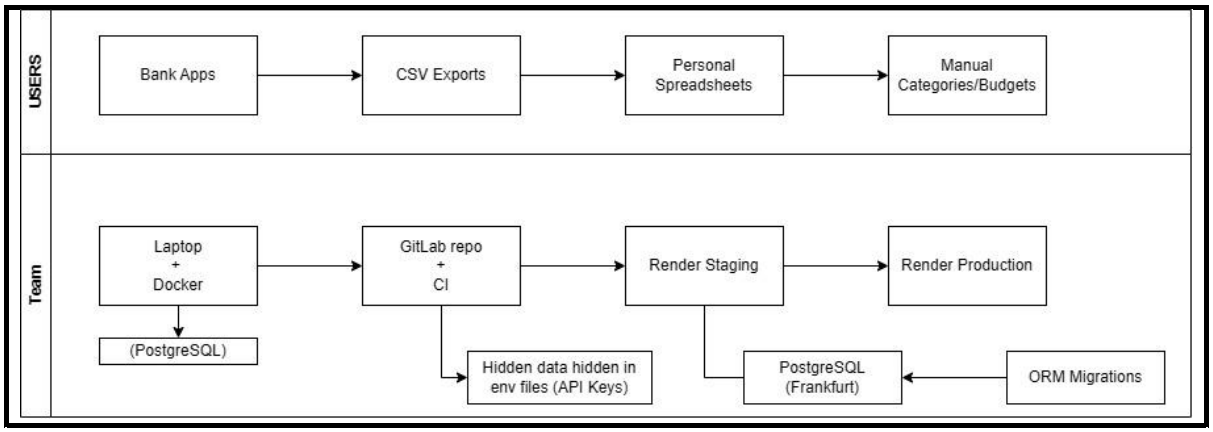
User rights: export data as CSV (access, portability), edit records (rectification), delete account (erasure), turn off AI (objection). Email requests are answered within one month.

If data is compromised: (1) contain it and reset exposed keys and passwords; (2) report to Ireland's Data Protection Commission within 72 hours if users are at risk; (3) inform affected users if the risk is high, and fix the root cause.

7. Migration

AIPFA is a new product for a start-up, so there is no existing infrastructure. Nothing needs to be lifted to the cloud or moved between different services. Noting this, we still need to upload the codebase from local machines to the cloud, existing financial data such as CSVs provided from banks and personal budgeting spreadsheets.

Dependency Tree



Dependency Tree created in Draw.io

Key Dependency: every feature is dependent on the database and user accounts (mentioned in the part 1 submission as FR01 and FR04)

Data to migrate and how it will be migrated.

Data	Source	Handling
Schema and default categories	Our code	Version Controlled ORM migrations run when application is deployed.
Users Transaction History	CSVs/XLSX provided from banks	CSVs are uploaded, validated, duplicate detection checks if the transaction have already been entered, categorised and appended to the database.
Accounts, Passwords	None	Users register on our service, and passwords are hashed with bcrypt. Nothing is migrated.